Atividade: Estratégias de Busca no Espaço de Estados

Resolução do Problema dos Jarros (BFS vs. DFS)

1. Definição do Problema e Representação de Estados

O clássico problema dos jarros consiste em dois recipientes sem marcação de medidas intermediárias:

- **Jarro A:** Capacidade de 4 litros.
- **Jarro B:** Capacidade de 3 litros.
- **Objetivo:** Obter exatamente 2 litros no Jarro A (Estado *(2, y)* para qualquer y).

Formalização Matemática do Espaço de Estados:

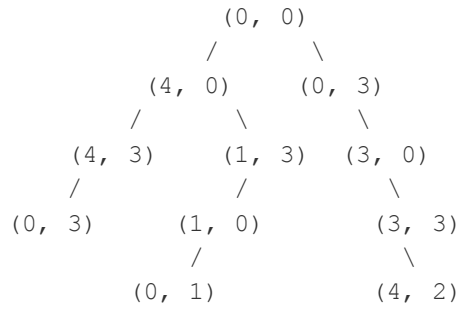
- **Estado:** Par ordenado (x, y), onde $x \in \{0, 1, 2, 3, 4\}$ e $y \in \{0, 1, 2, 3\}$.
- **Estado Inicial:** (0, 0)
- **Estado Objetivo:** (2, y), onde $y \in \{0, 1, 2, 3\}$.

Operadores de Transição (Ações):

1. Encher A (EA): (x, y) → (4, y) se x < 4
2. Encher B (EB): (x, y) → (x, 3) se y < 3
3. Esvaziar A (VA): (x, y) → (0, y) se x > 0
4. Esvaziar B (VB): (x, y) → (x, 0) se y > 0
5. Despejar A em B (A → B):
   - Se x + y ≤ 3 → (0, x + y)
   - Se x + y > 3 → (x - (3 - y), 3)
6. Despejar B em A (B → A):
   - Se x + y ≤ 4 → (x + y, 0)
   - Se x + y > 4 → (4, y - (4 - x))

2. Representação da Árvore de Estados

Abaixo está a estrutura hierárquica gerada a partir do nó raiz (0, 0), aplicando poda de repetição de estados no mesmo ramo de busca:





3. Simulação dos Algoritmos de Busca

A) Busca em Largura (Breadth-First Search - BFS)

A busca em largura utiliza uma fila FIFO para expandir todos os nós de um nível de profundidade antes de avançar para o próximo nível.

- **Nível 0:** Expandir (0,0) → Gera (4,0) e (0,3).
- **Nível 1:** Expandir (4,0) → Gera (4,3) e (1,3); Expandir (0,3) → Gera (3,0).
- **Nível 2:** Expandir (4,3) → Sem novos nós; Expandir (1,3) → Gera (1,0); Expandir (3,0) → Gera (3,3).
- **Nível 3:** Expandir (1,0) → Gera (0,1); Expandir (3,3) → Gera (4,2).
- **Nível 4:** Expandir (0,1) → Gera (4,1); Expandir (4,2) → Gera (0,2).
- **Nível 5:** Expandir (4,1) → Gera (2,3) [Meta Alcançada!]

Caminho Solução BFS (6 passos - Ótimo):
(0,0) →[EA]→ (4,0) →[A→B]→ (1,3) →[VB]→ (1,0) →[A→B]→ (0,1) →[EA]→ (4,1) →[A→B]→ (2,3)

B) Busca em Profundidade (Depth-First Search - DFS)

A busca em profundidade utiliza uma pilha LIFO, explorando o ramo mais profundo da árvore antes de realizar retrocesso (backtracking).

Ordem de expansão dos estados:
1. (0,0) → Expandir Encher A → (4,0)
2. (4,0) → Expandir Encher B → (4,3)
3. (4,3) → Esvaziar A → (0,3)
4. (0,3) → Despejar B em A → (3,0)
5. (3,0) → Encher B → (3,3)
6. (3,3) → Despejar B em A → (4,2)
7. (4,2) → Esvaziar A → (0,2)
8. (0,2) → Despejar B em A → (2,0) [Meta Alcançada!]

Caminho Solução DFS (8 passos):
(0,0) → (4,0) → (4,3) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0)

4. Comparativo de Desempenho e Estados Explorados

Critério de Avaliação	Busca em Largura (BFS)	Busca em Profundidade (DFS)
Estados Expandidos	9 estados	8 estados
Passos até a Solução	6 passos (Solução Ótima)	8 passos (Subótima)
Solução Encontrada	(2, 3)	(2, 0)
Garantia de Otimalidade	Sim (menor custo de passos)	Não (depende da ordem de regras)
Consumo de Memória	Alto (mantém toda a fronteira)	Baixo (apenas o ramo corrente)

Conclusão: A Busca em Largura (BFS) garante a solução com o menor número de operações possíveis (ótima em passos), porém exige maior retenção em memória. Já a Busca em Profundidade (DFS) encontra a solução rapidamente neste grafo específico com menor pegada de memória, mas encontrou um caminho com 8 passos em vez do ótimo de 6 passos.